

Ch10: Sorting Basics

Slide 1: Understanding Sorting

Before learning **Bubble Sort**, **Selection Sort**, or **Insertion Sort**, we first need to understand what **sorting** actually means.

Sorting means **arranging data in a particular order**.

Imagine a pile of books lying randomly on a table:

40 10 30 20

There is nothing wrong with the numbers, but they are not arranged in a useful order.

If we arrange them from the smallest value to the largest value:

10 20 30 40

the data is now **sorted in ascending order**.

Unsorted vs Sorted

Unsorted:

40 10 30 20

↓ ↓ ↓ ↓

Random arrangement

Sorted:

10 20 30 40

↑ ↑ ↑ ↑

Small → Large

The values themselves have not changed.

Only their **positions** have changed.

This is an important idea.

Before:

40 10 30 20

After:

10 20 30 40

We still have exactly the same four numbers:

10, 20, 30, 40

Sorting simply rearranges them.

A real-world analogy

Think about students standing randomly in a line.

Ravi → Anu → Kiran → Bala

If the teacher wants them arranged alphabetically:

Anu → Bala → Kiran → Ravi

Nobody disappeared. Nobody was duplicated.

The **order changed**.

Sorting an array works in the same basic way.

Key Idea

Sorting means rearranging existing data into a desired order. The data stays the same, but its positions change.

Slide 2: Why Do We Sort Data?

You might wonder:

If the values are already there, why bother rearranging them?

Because organized data is generally easier to work with.

Imagine trying to find the smallest number in:

67 12 89 23 5 45

You have to inspect the values and compare them.

After sorting:

5 12 23 45 67 89

the smallest value is immediately visible at the beginning, and the largest value is at the end.

Sorting can make data:

1. Easier to read

Compare:

Unsorted:

50 10 80 20 40

with:

Sorted:

10 20 40 50 80

The second arrangement is much easier for humans to understand.

2. Easier to analyze

Once values are organized, it becomes easier to identify:

Smallest value → First

Largest value → Last

For example:

15 20 35 48 90

↑

Min

↑

Max

3. Easier to search

Sorting is especially useful because some searching methods, such as **Binary Search**, require sorted data.

For example:

10 20 30 40 50 60 70

allows Binary Search to eliminate half of the search area at each step.

An unsorted array:

40 10 70 20 60 30 50

does not provide that same advantage.

So sorting and searching are often closely connected.

4. Easier to organize

Suppose marks are:

65 42 91 73 58

Sorting them gives:

42 58 65 73 91

Now the values are naturally organized from lowest to highest.

Key Idea

Sorting is not just about making numbers look neat. Organized data can make reading, analyzing, searching, and processing easier.

Slide 3: Understanding Ascending and Descending Order

There are two common ways to arrange numerical data:

1. **Ascending order**
2. **Descending order**

Ascending Order

Ascending means going from **smallest to largest**.

Think:

Low → High

Example:

10 20 30 40

Here:

$10 < 20$

$20 < 30$

$30 < 40$

Every value is greater than or equal to the value before it.

Visualize it as climbing stairs:

10
20
30
40

The values keep increasing.

Descending Order

Descending means going from **largest to smallest**.

Think:

High → Low

Example:

40 30 20 10

Here:

$40 > 30$

$30 > 20$

$20 > 10$

The values keep decreasing.

Visualize it as walking down stairs:

40
30

20
10

Comparing Both

Order	Direction	Example
Ascending	Small → Large	10 20 30 40
Descending	Large → Small	40 30 20 10

Consider:

Original:

30 10 40 20

Ascending:

10 20 30 40

Descending:

40 30 20 10

Notice that both sorted arrays contain the **same values**.

Only the direction of arrangement is different.

Important Connection to Algorithms

When we later write sorting programs, the condition we use determines the direction.

For ascending order, we want smaller values to move toward the beginning:

Small → Large

For descending order, we want larger values to move toward the beginning:

Large → Small

This small change in the comparison condition can change the entire result.

Key Idea

Ascending means smallest to largest. Descending means largest to smallest.

Slide 4: Sorting Numbers Step by Step

Let's take a slightly larger example:

Before sorting:

25 5 40 10 15

Our goal is **ascending order**:

5 10 15 25 40

At first, the values are mixed:

25 5 40 10 15

We need to rearrange their positions.

Think of sorting as gradually putting values into their correct locations.

Step 1: Identify a value that belongs near the beginning

The smallest value is:

5

So eventually, 5 should be at the beginning:

5 _ _ _ _

The remaining values are:

10 15 25 40

Step 2: Place the next smallest value

The next smallest value is:

10

So:

5 10 _ _ _

Step 3: Place the next value

The next smallest is:

15

Now:

5 10 15 _ _

Step 4: Continue

The remaining values are:

25 40

So:

5 10 15 25 40

Now the array is sorted.

The important part

A sorting algorithm has to figure out **how to move values into these positions**.

That is where algorithms such as Bubble Sort, Selection Sort, and Insertion Sort come in.

They all have the same general goal:

Unsorted Data



Compare values



Move/rearrange values



More comparisons



More rearrangement



Sorted Data

But they do this differently.

For example:

Bubble Sort

→ repeatedly compares neighboring elements

Selection Sort

→ repeatedly selects the smallest remaining element

Insertion Sort

→ takes an element and inserts it into its correct position

the important thing at this stage is **not** to memorize those algorithms yet.

First understand the common goal:

Start with disorganized data and systematically rearrange it until the required order is achieved.

Slide 5: Sorting in C

C does not automatically look at an ordinary integer array and decide:

"These numbers look messy. I'll organize them."

Human programmers have to provide the instructions. Humanity apparently decided that even arranging five numbers should require an algorithm.

Consider:

```
int numbers[] = {25, 5, 40, 10, 15};
```

The array is stored exactly as we give it:

25 5 40 10 15

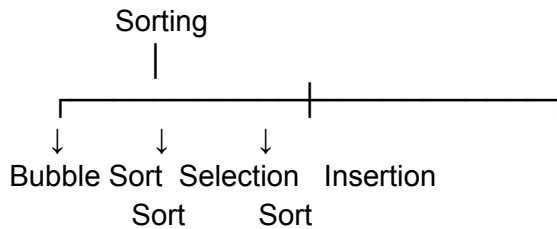
C will not automatically change it to:

5 10 15 25 40

We need to write an algorithm that performs the rearrangement.

Common sorting algorithms

Three important sorting algorithms we will study are:



Bubble Sort

Bubble Sort repeatedly compares **neighboring elements**.

25 5 40 10 15

↑ ↑

Compare

If they are in the wrong order, they are swapped.

Selection Sort

Selection Sort looks for the appropriate value, such as the **smallest remaining value**, and places it into its correct position.

25 5 40 10 15

↑

Smallest

It gradually builds the sorted portion of the array.

Insertion Sort

Insertion Sort takes one value at a time and inserts it into the correct position among the values already arranged.

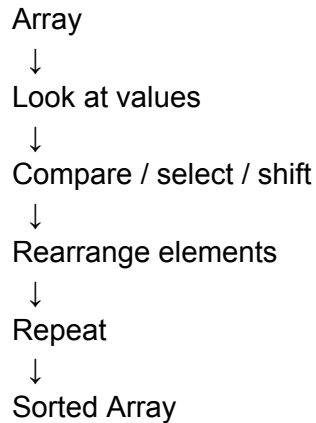
Sorted part | Unsorted part

5 25 | 40 10 15

Then the next value is inserted where it belongs.

Common structure

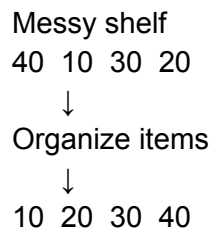
Although these algorithms work differently, they all follow the same broad idea:



The difference is **how they decide which elements to compare and how they move them.**

Final Mental Model

Think of sorting as organizing a messy shelf.



The three algorithms are simply **different strategies for doing that organization.**

Bubble Sort

→ Compare neighbors

Selection Sort

→ Select the correct value

Insertion Sort

→ Insert into the correct position

Core Takeaway

Sorting means arranging data into a required order. Ascending goes from small to large, while descending goes from large to small. In C, sorting an ordinary array requires an algorithm, and Bubble Sort, Selection Sort, and Insertion Sort are three important ways to perform that rearrangement.